

LISTA 3b DE EXERCÍCIOS – Comunicação e Sincronização entre Processos – Semáforos e Troca de Mensagens

1. Explique como podem ser implementadas as primitivas P(s) e V(s) do mecanismo de semáforos.
2. Explique como os semáforos podem ser usados na implementação da exclusão mútua e da sincronização baseada em condições. Quais os tipos de semáforos mais adequados a estas implementações?
3. Explique como os semáforos podem ser usados na implementação do controle da alocação de recursos N-plicados.
4. Explique o mecanismo de troca de mensagens. Na troca de mensagens, como a sincronização entre processos é garantida?
5. Explique o funcionamento das primitivas de troca de mensagens:
 - a. Envia bloqueante e Recebe bloqueante
 - b. Envia não bloqueante e Recebe não bloqueante
6. As primitivas de troca de mensagens podem ser usadas na comunicação e sincronização entre processos que estão executando em um mesmo computador? Explique.

Lista de Exercícios 3b - Semáforos e Troca de Mensagens

Nome: Igor dos Reis Gomes

RA: 241025265

1- Para todo semáforo existe uma fila q_i associada a este semáforo S_i .

A primitiva $P(S_i)$ faz a verificação se é possível utilizar do recurso.

Se $S_i > 0$, disponível e $S_i = S_i - 1$, processo continua

Se $S_i = 0$, ocupado, processo bloqueado e colocado na fila q_i

A primitiva $V(S_i)$ sinaliza a liberação do recurso.

Se $S_i = 0$, $S_i = S_i + 1$

Se $S_i > 0$, acorda um processo da fila q_i

2- Na exclusão mútua, a utilização de semáforos garante que apenas um processo acesse a região crítica. Antes de entrar na RC o processo chama $P(\text{mutex})$ e ao sair chama $V(\text{mutex})$. Para esse tipo de implementação, o semáforo mais adequado é o binário (iniciando com valor 1).

Na sincronização baseada em condições, um processo deve esperar até que uma condição seja satisfeita. O processo executa $P(s)$ e bloqueia se a condição não for atendida ($s = 0$) e continua se $s > 0$, além disso, se um processo torna uma condição verdadeira, este sinaliza usando $V(s)$, fazendo com que se q_s não estiver vazia, vai desbloquear um processo e caso estiver vazia, irá incrementar s .

3- Para realizar esse controle de início o semáforo é inicializado com valor de N (número de recursos disponíveis). Para obter um desses recursos executa $P(s)$, enquanto houver recursos ($s > 0$), os processos conseguem passar e quando $s = 0$ todos os N recursos estão em uso bloqueando o próximo processo que chama $P(s)$. Na liberação, o processo que terminou de usar o recurso executa $V(s)$ e, caso não tenha nenhum outro processo requisitando o recurso, sinaliza que existe um recurso disponível ($s = s + 1$) ou acorda o processo que estava na fila para usar o recurso que acabou de ser liberado.

4- A troca de mensagens é uma forma de comunicação entre processos sem o uso de variáveis compartilhadas. Em vez de memória compartilhada, os processos se comunicam a partir das primitivas $\text{envia}(\text{mensagem}, \text{destinatario})$ e $\text{recebe}(\text{mensagem}, \text{remetente})$, sendo a mensagem uma área da memória para transportar dados. A sincronização é garantida pelo próprio ato de esperar a mensagem, é estritamente necessário que a mensagem tenha sido enviada primeiro.

5-

a) No envio bloqueante, o remetente fica parado até que tenha-se um sinal que a mensagem foi recebida e no recebimento bloqueante, o receptor fica bloqueado até que uma mensagem chegue a ele.

b) No envio não bloqueante o processo envia a mensagem e continua sua execução, sem saber se a mensagem foi recebida e no recebimento não bloqueante, verifica se há mensagem, se tiver ele lê, mas se não tiver continua sua execução.

6- Sim, é perfeitamente possível a troca de mensagens em um mesmo computador, porém irá existir um custo maior de memória e de tempo do que utilizando variáveis compartilhadas, já que o SO precisa copiar a mensagem da memória do processo que está enviando para a área do SO e depois para o processo que está recebendo a mensagem.